Teaching Philosophy

Waseem Raza, PhD | Assistant Professor of Computer Science | Susquehanna University

Helping students explain what their code is doing

When students first learn to program, it is easy for them to see a successful output as proof that they understand a solution. I try to lead them further: What problem is the program solving? What assumptions does it make? What happens with an unexpected input? How could another person test or maintain it? These questions matter in introductory programming just as much as in a senior capstone. They also reflect the way I approach research, where a good result is only useful if I understand the conditions under which it holds.

At the University of Lahore, I taught Computer Programming, Communication Systems, and Wireless Communication. I also supported engineering laboratories and course assessment at the University of Engineering and Technology, Taxila, and mentored Bachelor's and Master's students in communications-related research at COMSATS University Islamabad. Outside university teaching, I have tutored younger students, including those at the tenth- and twelfth-grade levels. Teaching across these settings has helped me pay attention to the point at which a student loses the thread of an explanation. A small gap in a prerequisite concept can become a much larger obstacle later, especially when programming and mathematics are learned together.

My approach has three recurring elements: make reasoning visible, give students room to test ideas, and use feedback while it can still change their work.

From a worked example to an independent decision

I would build an introductory programming lesson around short exercises that students can reason about before running any code. For example, students could trace a loop that processes a list of measurements, predict the result for an empty list or an unexpected value, and compare their reasoning with a small set of tests. A later assignment might ask them to implement and compare two approaches to finding patterns in data, documenting both correctness and computational cost. I would use such exercises to connect syntax to problem decomposition, data representation, algorithmic thinking, and debugging.

My interests align most directly with foundational programming and computing courses, data-oriented computation, computer and communication networks, and applied machine learning. Susquehanna's curriculum includes Principles of Computer Science, Object Oriented Programming and Design, Data Structures, and Algorithms. I would prepare course-specific examples and assessments in consultation with colleagues and departmental outcomes, while maintaining the conceptual rigor expected in these foundational subjects. Where courses introduce specialized systems or theory beyond my prior instructor-of-record subjects, I would use deliberate preparation rather than assume expertise merely because the topics are adjacent to my research.

For advanced undergraduates, I would favor assignments with a reproducible starting point and an open decision to make. In a networks or data-science class, a student might analyze a supplied dataset, implement a simple baseline, change one design choice, and show whether the evidence supports the change. The point is not to make every exercise an ML project. It is to develop habits of careful measurement, explanation, and revision that transfer from one field of computing to another.

Assessment and a classroom where students can participate

I consider assessment part of instruction. Frequent, low-stakes checks can reveal misunderstandings about variables, control flow, functions, data structures, or model evaluation before they are reinforced by larger assignments. I would use short coding tasks, written explanations, and code reviews alongside conventional quizzes and exams. Rubrics would reward readable design, appropriate tests, reasoning about limitations, and communication, not simply a correct final output.

Students enter computing with different degrees of familiarity. Some have built programs for years; others may be encountering a command line or debugger for the first time. I would begin by making the tools and prerequisites explicit, sharing a tested starter environment where useful, and creating structured opportunities to ask questions. Pair or group activities can help, but I would rotate technical and explanatory roles so that the most experienced programmer does not do all the work. Office hours, written feedback, and short follow-up exercises should be normal parts of the course rather than emergency measures after a poor exam.

I also want students to ask where technical systems obtain their data, who might be affected by an incorrect prediction, and whether the measured improvement matters outside a classroom benchmark. An activity-recognition model is a useful example: even a technically accurate classifier can fail for a user whose movement differs from the training data. Discussing these cases makes fairness, privacy, and reproducibility concrete without displacing core CS content.

Mentoring individual projects and capstones

Individual capstones provide a different opportunity from ordinary homework. A student can choose a question, read relevant work, design an approach, and learn to manage uncertainty when there is no answer key. I would begin a project with a scope conversation: what can be completed in a semester or academic year with the student's current preparation? We would write down the research or engineering question, a realistic baseline, the data and tools required, milestones, and a definition of success that includes an honest account of negative findings.

A student interested in networking could study anomaly detection on publicly available network traces. A student drawn to data science might evaluate why a classifier deteriorates when its test distribution changes. Another might develop a small sensor-data analysis tool and document which activities it confuses. I would meet regularly, use a shared version-controlled repository and brief progress notes, and support students in deciding when to simplify a design. Finished work should include readable code, a reproducible experiment, clear limitations, and a presentation suitable for a broader undergraduate audience. Publication can be a worthwhile outcome, but it would not be the only measure of a successful capstone.

Teaching and learning as an ongoing practice

Susquehanna's undergraduate focus appeals to me because it gives faculty the opportunity to know students as learners and to see their growth across multiple courses. I would look for patterns in student work, solicit specific midsemester feedback, consult colleagues about course progression, and revise assignments based on what students actually learn. I want graduates to leave not only with programming and analytical skills, but with the confidence to explain their decisions, the patience to test an assumption, and the judgment to recognize a result that should not yet be trusted.